

Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών
Τμήμα Πληροφορικής και Τηλεπικοινωνιών

ΠΑΡΑΛΛΗΛΑ ΣΥΣΤΗΜΑΤΑ

Εργασία 1 - Προγραμματισμός με Pthreads

Ονοματεπώνυμο : Κυριακάκος Νικόλαος — Αριθμός Μητρώου : 1115202200083

Ονοματεπώνυμο : Γαυγιωτάκη Ευθυμία — Αριθμός Μητρώου : 1115202200027

Περιεχόμενα

1 Δομή Κώδικα **2**

2 Εκτέλεση προγραμμάτων **3**
2.1 Μετρικές . . . 3
2.2 Αρχείο tests.sh . . . 3
2.3 Συμβάσεις κώδικα . . . 3

3 Άσκηση 1.1 **4**
3.1 Βήματα εκτέλεσης . . . 4
3.2 Αποτελέσματα εκτέλεσης . . . 4

4 Άσκηση 1.2 **5**
4.1 Βήματα εκτέλεσης . . . 5
4.2 Αποτελέσματα εκτέλεσης . . . 5

5 Άσκηση 1.3 **7**
5.1 Βήματα εκτέλεσης . . . 7
5.2 Αποτελέσματα εκτέλεσης . . . 7

6 Άσκηση 1.4 **9**
6.1 Τροποποιήσεις δοσμένου κώδικα . . . 9
6.2 Προσθήκη κώδικα . . . 9
6.3 Αποτελέσματα εκτέλεσης . . . 9

1 Δομή Κώδικα

Ο κώδικας αποτελείται από τους εξής φακέλους:

- Bin: εδώ τοποθετούνται τα εκτελέσιμα αρχεία που προκύπτουν από τη μεταγλώττιση των προγραμμάτων
- Build: εδώ τοποθετούνται τα αντικείμενα αρχεία που προκύπτουν από τη μεταγλώττιση των προγραμμάτων
- Results: εδώ τοποθετούνται τα αρχεία που περιέχουν τα αποτελέσματα του προγραμμάτων, τα γραφήματα καθώς και ένα αρχείο που περιέχει τις πληροφορίες του συστήματος που ζητούνται από την εκφώνηση.
- Include: στον φάκελο αυτό υπάρχουν τα αρχεία κεφαλίδας
- Src: στον φάκελο αυτό βρίσκεται ο πηγαίος κώδικας των προγραμμάτων

Περιλαμβάνονται επίσης τα εξής αρχεία:

- Makefile
- graphs.py: το πρόγραμμα αυτό χρησιμοποιείται για να διαβάσει και να αποθηκεύσει κατάλληλα τα αποτελέσματα που βρίσκονται στα αρχεία που δημιουργήθηκαν από την εκτέλεση των προγραμμάτων και να δημιουργήσει τα αντίστοιχα γραφήματα.
- tests.sh: το αρχείο αυτό είναι εκείνο το οποίο χρησιμοποιήθηκε για τη δημιουργία και εκτέλεση των προγραμμάτων.

2 Εκτέλεση προγραμμάτων

2.1 Μετρικές

Τα προγράμματα δοκιμάστηκαν στο μηχάνημα linux19 της σχολής.
Το μηχάνημα αυτό έχει 4 πυρήνες. Συνεπώς όπως αναφέρθηκε και στο μάθημα, τα τεστ που διενεργήθηκαν έγιναν με αριθμό νημάτων μέχρι 4 * πυρήνες, δηλαδή 16.
Συνεπώς, σε όλα τα τεστ (και στα αποτελέσματα που παρατίθενται στη συνέχεια) έχουν χρησιμοποιηθεί οι αριθμοί νημάτων 1 2 4 8 16.
Κάθε πρόγραμμα εκτελέστηκε με διαφορετικό ορισμάτων 4 φορές το καθένα. Αυτό είχε ως σκοπό να μετρηθεί ο μέσος όρος των αποτελεσμάτων ώστε να αποφευχθούν διάφορες αστοχίες. Τα αποτελέσματα που ακολουθούν παρακάτω είναι όλα αυτά των μέσων όρων. Εκτελώντας ωστόσο το αρχείο tests.sh προκύπτουν επίσης όλα τα αποτελέσματα αναλυτικά στο φάκελο results.
Τα χαρακτηριστικά του υπολογιστικού συστήματος που χρησιμοποιήθηκε (linux19) είναι τα εξής:

Χαρακτηριστικό	**Λεπτομέρειες**
Hostname	linux19
CPU Model	Intel(R) Core(TM) i5-6500 CPU @ 3.20GHz
CPU Cores	4
OS Version	Ubuntu 20.04.6 LTS
GCC Version	gcc (Ubuntu 9.4.0-1ubuntu1 20.04.2) 9.4.0

2.2 Αρχείο tests.sh

Για την εκτέλεση των προγραμμάτων και στη συνέχεια τη δημιουργία των γραφημάτων χρησιμοποιήθηκε το αρχείο tests.sh.
Ουσιαστικά μέσω των συναρτήσεων που περιέχει κάνει τα εξής:

1. καθαρίζει τυχόν προηγούμενα αποτελέσματα εκτελέσεων και μεταγλωττίσεων
2. μέσω εντολών bash μας δίνει τις πληροφορίες του συστήματος που εκτελείται το πρόγραμμα (βρίσκονται στο αρχείο system_info.txt στον φάκελο results)
3. μεταγλωττίζει τον πηγαίο κώδικα και δημιουργεί τα αντίστοιχα εκτελέσιμα προγράμματα
4. εκτελεί τον κώδικα του αρχείου graphs.py ο οποίος παράγει τα γραφήματα που επιθυμούμε

2.3 Συμβάσεις κώδικα

1. Υπάρχει ένα αρχείο common.c στον φάκελο src, στο οποίο υπάρχουν δύο συναρτήσεις διαχείρισης λάθους. Αυτές εμφανίζουν το λάθος στο χρήστη (αναλόγως αν είναι λάθος συστήματος ή λάθος για παράδειγμα ορισμάτων) και καλώντας της συνάρτηση exit τερματίζουν.
2. Ο κώδικας για κάθε άσκηση βρίσκεται στο αρχείο με όνομα αυτό της αντίστοιχης άσκησης (πχ 1_1.c) εκτός από τις άσκησης 1.4 η οποία χρησιμοποιεί τον κώδικα που βρίσκεται στον φάκελο με όνομα 1_4.
3. Χρησιμοποιείται το αρχείο tests.sh για εκτέλεση όλων των προγραμμάτων ή εφόσον έχουν μεταγλωττιστεί τα προγράμματα, καλούνται με εντολή όπως η ./bin/1_1
4. Τα αρχεία αποτελεσμάτων έχουν όνομα που ξεκινάει με το όνομα του προγράμματος ακολουθούμενο από _table.txt.
   Ομοίως, τα γραφήματα έχουν αντίστοιχο όνομα με αυτό των προγραμμάτων με κατάληξη _graph.png.

3 Άσκηση 1.1

Η συγκεκριμένη άσκηση έχει ως σκοπό τη χρήση της μεθόδου Monte Carlo προκειμένου να εκτιμηθεί η τιμή του π με σειριακό αλγόριθμο και στη συνέχεια με χρήση νημάτων.

3.1 Βήματα εκτέλεσης

1. Ελέγχεται ότι τα ορίσματα είναι σωστά.
   Συγκεκριμένα, δέχεται ως πρώτο όρισμα των αριθμό των νημάτων που θα χρησιμοποιηθούν. και ως δεύτερο όρισμα δέχεται τον αριθμό των ρίψεων. Ελέγχεται ότι και τα δύο ορίσματα είναι θετικοί αριθμοί.

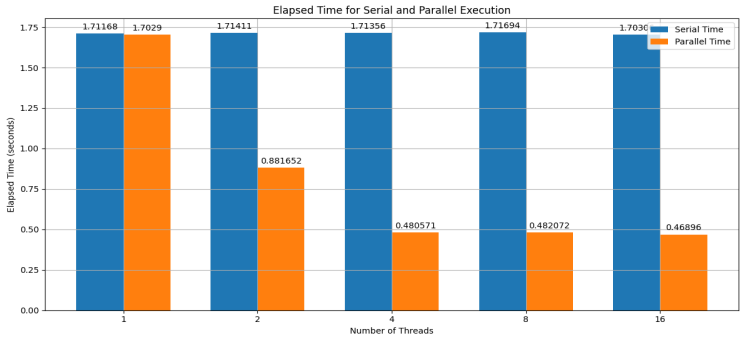
2. Το πρόγραμμα υπολογίζει την εκτίμηση για την τιμή του π χρησιμοποιώντας τον σειριακό αλγόριθμο που περιγράφεται στην εκφώνηση

3. Με χρήση του παράλληλου αγοριθμού που περιγράφεται υπολογίζεται η τιμή του π

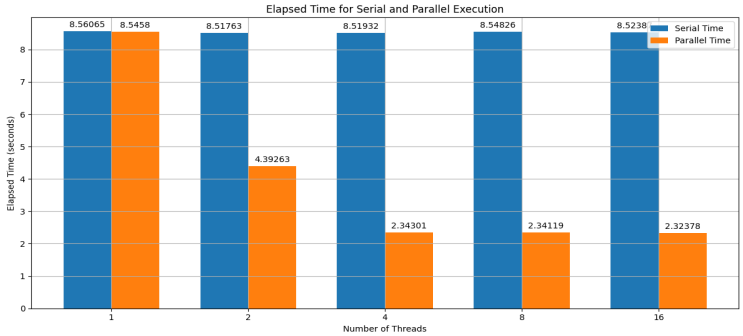
4. Εκτυπώνονται τα αποτελέσματα τόσο στην οθόνη όσο και σε ένα αρχείο αποτελεσμάτων

3.2 Αποτελέσματα εκτέλεσης

Στην άσκηση αυτή δεν χρειάστηκε να χρησιμοποιηθεί συγχρονισμός και αυτό διότι κάθε νήμα στο παράλληλο υπολογισμό του π χρησιμοποιεί τα δικά του δεδομένα και συνεπώς δεν υπάρχει κάποιος κίνδυνος.
Παρατηρούμε ότι αυξάνοντας τον αριθμό των νημάτων γίνεται πιο γρήγορα ο υπολογισμός, γεγονός το οποίο είναι απολύτως φυσιολογικό, καθώς κάθε νήμα αναλαμβάνει ισάξιο όγκο υπολογισμών κι έτσι επιτυγχάνεται ο καταμερισμός εργασίας στο πρόγραμμα.
Παρακάτω, φαίνεται ο μέσος όρος των αποτελεσμάτων 4 εκτελέσεων για την άσκηση αυτή με αριθμούς νημάτων αυτούς που φαίνονται και 100000000 (10^8) ρίψεις.



Στο ακόλουθο σχήμα παρουσιάζεται το αποτέλεσμα για ίδιους αριθμούς νημάτων και 500000000 ($5 * 10^8$) ρίψεις.



4 Άσκηση 1.2

4.1 Βήματα εκτέλεσης

1. Ελέγχεται ότι τα ορίσματα είναι σωστά.
   Συγκεκριμένα, δέχεται ως πρώτο όρισμα των αριθμό των νημάτων που θα χρησιμοποιηθούν. και ως δεύτερο όρισμα δέχεται τον αριθμό των επαναλήψεων. Ελέγχεται ότι και τα δύο ορίσματα είναι θετικοί αριθμοί.
   Έπειτα, δέχεται τον αριθμό 0 αν πρόκειται να χρησιμοποιηθούν ατομικές εντολές αλλιώς τον αριθμό 1 αν πρόκειται να χρησιμοποιηθεί mutex.

2. Εφόσον έχει δοθεί ως επιλογή να χρησιμοποιηθεί mutex, αρχικοποιείται.

3. Δημιουργείται ο αριθμός των νημάτων τον οποίο επιθυμεί ο χρήστης και δέχεται ως όρισμα την αντίστοιχη δομή που χρησιμοποιείται για κάθε τύπο νήματος

4. Γίνεται η αύξηση της κοινώς της μεταβλητής με τον τρόπο τον οποίο έχει διαλέξει ο χρήστης και επιστρέφεται η τιμή της

5. Καταστρέφεται το mutex εφόσον ο χρήστης έχει επιλέξει αυτή τη μέθοδο

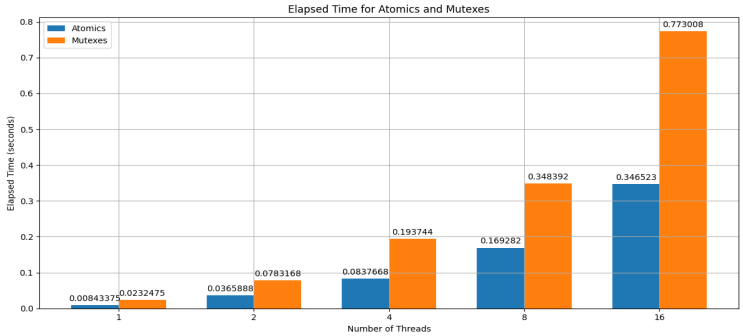
6. Εκτυπώνονται τα αποτελέσματα τόσο στην οθόνη όσο και σε ένα αρχείο αποτελεσμάτων

4.2 Αποτελέσματα εκτέλεσης

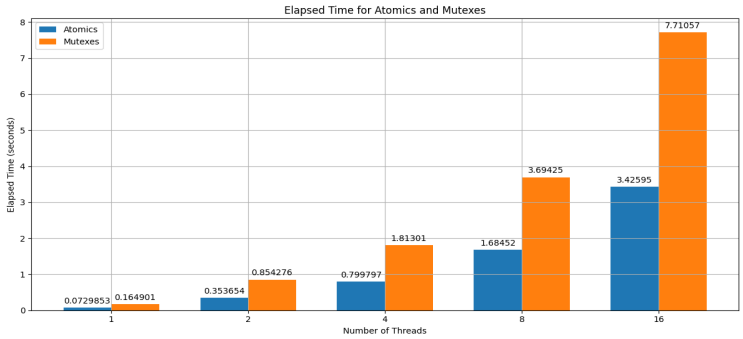
Παρατηρούμε ότι σε όλες τις περιπτώσεις με τη χρήση των ατομικών εντολών το πρόγραμμα τερματίζει πολύ πιο γρήγορα από ότι σε σχέση με τη χρήση mutex.
Αυτό συμβαίνει καθώς ατομικές εντολές είναι πολύ πιο γρήγορες επειδή εκτελούνται σε επίπεδο hardware χωρίς την ανάγκη συστήματος κλήσεων ή μηχανισμών διαχείρισης όπως τα mutex.
Το γεγονός αυτό εξαλείφει το overhead του context switching, τις καθυστερήσεις λόγω blocking, και την ανάγκη για συντονισμό μεταξύ νημάτων.
Αντίθετα, τα mutex προκαλούν επιπλέον καθυστερήσεις λόγω αναμονής και επικοινωνίας μεταξύ των νημάτων, κάνοντας τις ατομικές εντολές ιδανικές για απλές, συχνές πράξεις όπως η αύξηση ενός μετρητή.
Τέλος, παρατηρούμε ότι όσο περισσότερα νήματα έχουμε, τόσο πιο αργά εκτελείται το πρόγραμμα, γεγονός που δικαιολογείται από τα παραπάνω και από το ότι όλα τα νήματα επενεργούν σε μια κοινόχρηστη μεταβλητή.

Στην επόμενη σελίδα βλέπουμε και τα σχήματα τα οποία επιβεβαιώνουν τις παραπάνω παρατηρήσεις.

Παρακάτω, φαίνεται ο μέσος όρος των αποτελεσμάτων 4 εκτελέσεων για την άσκηση αυτή με αριθμούς νημάτων αυτούς που φαίνονται και 1000000 (10^6) επαναλήψεις.



Στο ακόλουθο σχήμα παρουσιάζεται το αποτέλεσμα για ίδιους αριθμούς νημάτων και 10000000 (10^7) επαναλήψεις.



5 Άσκηση 1.3

5.1 Βήματα εκτέλεσης

1. Ελέγχεται αν ο αριθμός των νημάτων και των επαναλήψεων που δέχεται το πρόγραμμα ως ορίσματα είναι θετικοί αριθμοί. Επίσης ελέγχεται άν το τελευταίο δοσμένο argument αντιστοιχεί σε 0 ή 1 δηλαδή σε μία από τις δύο μεθόδους υλοποίησης.

2. Ορίζεται ένας κοινόχρηστος πίνακας αρχικοποιημένος με 0

3. Χωρίζονται οι συνολικες επαναληψεις στα νήματα, και κάθε νήμα εκτελεί τη συνάρτηση στην οποία ενημερώνει ένα μόνο στοιχείο του κοινόχρηστου πίνακα, το οποίο βρίσκεται στη θέση που αντιστοιχεί στον αριθμό του thread

4. Προσθέτονται σε μία μετβλητή όλες οι τιμές του τελικού πίνακα και εκτυπώνονται τα αποτελέσματα στον τερματικό και στο αρχείο αποτελεσμάτων.

5.2 Αποτελέσματα εκτέλεσης

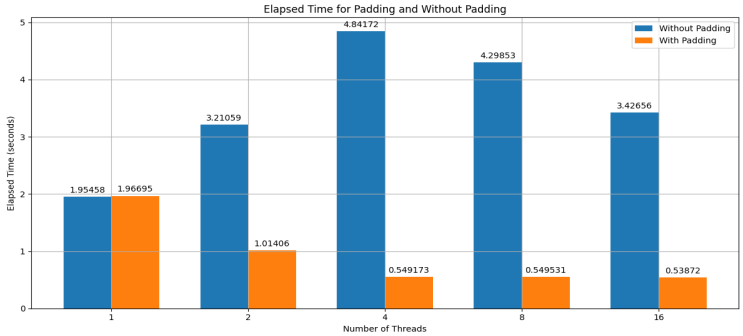
Στην συγκεκριμένη υλοποίηση δεν χρειάστηκε η χρήση συγχρονισμού καθώς το κάθε νήμα επεξεργάζεται δεδομένα διαφορετικών μεταβλητών που βρίσκονται σε διαφορετικές θέσεις του πίνακα και θέσεις μνήμης. Επομένως δεν δημιουργούνται race conditions, και δεν υπάρχει κίνδυνος λάθος αποτελεσμάτων.

Υπάρχουν στον κώδικα για την επίλυση της άσκησης δύο υλοποιήσεις. Μία με padding και μία χωρίς. Αν επιλεχθεί να εκτελεστεί χωρίς padding τότε παρατηρούμε οτι όσο ανεβαίνει ο αριθμός των νημάτων από 1 σε 4 δεν υπάρχει βελτίωση στον χρόνο εκτέλεσης, αντιθέτως γίνεται πιο αργός και αρχίζουμε να βλέπουμε βελτίωση από την χρήση 8ου νήματος και πάνω. Αυτό οφείλεται στο φαινόμενο false sharing που εμφανίζεται όταν πολλά νήματα θέλουν να επεξεργαστούν δεδομένα από διαφορετικές μεταβλητές που είναι όμως αποθηκευμένα κοντά στη μνήμη ή συγκεκριμένα στο ίδιο cache line. Όταν ένα νήμα αποκτά πρόσβαση σε μια μεταβλητή, φορτώνεται ολόκληρη η γραμμή cache στην οποία βρίσκεται και προκαλούνται καθυστερήσεις στον συγχρονισμό και στην επικοινωνία μεταξύ των πυρήνων. Η λύση σε αυτό το πρόβλημα είναι η ευθυγράμμιση δεδομένων, δηλαδή η δεύτερη μέθοδος στον κώδικα. Allignment επιτυγχάνεται με padding, δηλαδή με τη χρήση κενών δεδομένων ώστε τα στοιχεία να βρίσκονται σε διαφορετικές γραμμές cache.

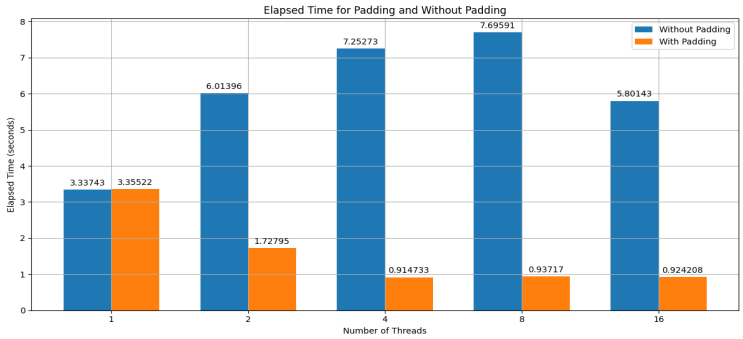
Έτσι βλέπουμε τα επιθυμητά αποτελέσματα στη βελτίωση χρόνου όσο αυξάνεται ο αριθμός των νημάτων καθώς το ίδιο φόρτο εργασίας μοιράζεται σε πολλά νήματα που εκτελούν τις ενημερώσεις τους παράλληλα χωρίς να εμποδίζονται μεταξύ τους.

Στην επόμενη σελίδα βλέπουμε και τα σχήματα τα οποία επιβεβαιώνουν τις παραπάνω παρατηρήσεις.

Παρακάτω, φαίνεται ο μέσος όρος των αποτελεσμάτων 4 εκτελέσεων για την άσκηση αυτή με αριθμούς νημάτων αυτούς που φαίνονται και 1000000000 (10^9) ενημερώσεις.



Στο ακόλουθο σχήμα παρουσιάζεται το αποτέλεσμα για ίδιους αριθμούς νημάτων και 5000000000 ($5 * 10^7$) ενημερώσεις.



6 Άσκηση 1.4

Στην άσκηση αυτή δημιουργούμε ένα κλείδωμα ανάγνωσης-εγγραφής (read write lock).
Το κλείδωμα αυτό υλοποιεί δύο προσεγγίσεις:

- στην πρώτη προσέγγιση θα δίνεται προτεραιότητα στα νήματα ανάγνωσης
- στην δεύτερη προσέγγιση θα δίνεται προτεραιότητα στη νήματα εγγραφής

6.1 Τροποποιήσεις δοσμένου κώδικα

Για τη συγκεκριμένη άσκηση χρησιμοποιούμε τα δοσμένα αρχεία my_rand.h, my_rand.c, timer.h, pth_ll_rwl.c.
Από τα παραπάνω αρχεία έχει τροποποιηθεί μόνο το pth_ll_rwl.c ως εξής:

- Έχει προστεθεί ως global μεταβλητή το νέο κλείδωμα
- Έχει προστεθεί μία νέα συνάρτηση η οποία εκτυπώνει τα αποτελέσματα σε ένα αρχείο
- Αντί να χρησιμοποιείται η συνάρτηση scanf προκειμένου χρήστης να δώσει τις τιμές που επιθυμεί αυτές λαμβάνονται ως ορίσματα
- Έχει προστεθεί η εντολή δημιουργίας και καταστροφής του κλειδώματος
- Έχει τροποποιηθεί η συνάρτηση Thread_work, η οποία ουσιαστικά πλέον δοσμένου του ορίσματος εκτέλεσης αν πρόκειται για readers η writers προτεραιότητα, καλεί της κατάλληλες συναρτήσεις του κλειδώματος
- Έχει τροποποιηθεί η συνάρτηση Usage έτσι ώστε να εμφανίζει σωστά το μήνυμα λάθους για τα ορίσματα του προγράμματος

6.2 Προσθήκη κώδικα

Έχουν προστεθεί δύο νέα αρχεία (custom_rw_lock.h και custom_rw_lock.c)
Θεωρούμε ότι μπορεί να υπάρχει μόνο ένας writer ενεργός (γι'αυτό και χρησιμοποιούμε την boolean μεταβλητή writer_active). Σε όλα τα άλλα μέλη της δομής χρησιμοποιούμε αρίθμηση, καθώς μπορεί για παράδειγμα να περιμένουν πολλά νήματα-αναγνώστες.
Οι συναρτήσεις που έχουν δημιουργηθεί ανήκουν σε 3 κατηγορίες:

- Αυτές οι οποίες είναι κοινές συναρτήσεις, δηλαδή αυτές τις αρχικοποίησης και της καταστροφής
- Συναρτήσεις οι οποίες δίνουν προτεραιότητα στους readers
- Συναρτήσεις οι οποίες δίνουν προτεραιότητα στους writers

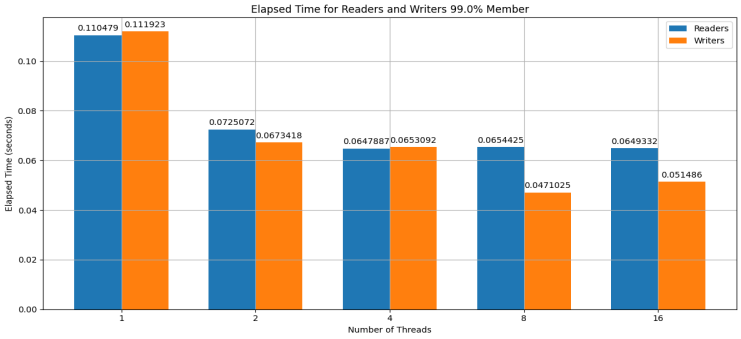
Αναλόγως το όρισμα που δίνεται στην εκτέλεση του προγράμματος κρίνεται που επιθυμεί ο χρήστης να δώσει προτεραιότητα (0 για προτεραιότητα στους readers και 1 για writers).

6.3 Αποτελέσματα εκτέλεσης

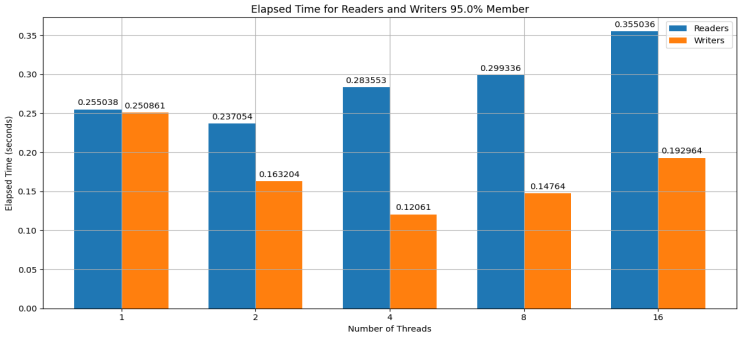
Παρατηρούμε σε όλες τις περιπτώσεις οτι ο χρόνος εκτέλεσης του προγράμματος με προτεραιότητα στα νήματα ανάγνωσης είναι μικρότερος από αυτόν με προτεραιότητα στα νήματα εγγραφής. Όταν δίνουμε προτεραιότητα στους readers, πολλά νήματα μπορούν να διαβάσουν τα δεδομένα ταυτόχρονα και έχουν μεγάλο βαθμό παραλληλίας. Σε όλες τις εκτελέσεις μας ο αριθμός των readers είναι σημαντικά μεγαλύτερος απο τον αριθμό των writers. Συνεπώς η καθυστερήσεις που προκύπτουν από την αναμονή των νημάτων εγγραφής είναι ασήμαντες. Αντίθετα δίνοντας προτεραιότητα στους writers και αναγκάζοντας όλα τα νήματα ανάγνωσης να περιμένουν κάθε φορά, οδηγούμαστε σε μεγάλες καθυστερησεις. Οι writers δουλεύουν ουσιασικά σειριακά, δεν αξιοποιείται μέγιστα ο παραλληλισμός και υπάρχει κίνδυνος αδρανοποίησης των readers.

Στα ακόλουθα σχήματα χρησιμοποιούμε 1000 (10^3) κλειδιά και 50000 ($5 * 10^4$) λειτουργίες και αλλάζουμε τα ποσοστά της Member.

Παρακάτω, φαίνεται ο μέσος όρος των αποτελεσμάτων 4 εκτελέσεων για την άσκηση αυτή με αριθμούς νημάτων αυτούς που φαίνονται, ποσοστό 99% αναζήτησης και 1% εισαγωγής.



Παρακάτω, φαίνεται ο μέσος όρος των αποτελεσμάτων 4 εκτελέσεων για την άσκηση αυτή με αριθμούς νημάτων αυτούς που φαίνονται, ποσοστό 95% αναζήτησης και 5% εισαγωγής.



Παρακάτω, φαίνεται ο μέσος όρος των αποτελεσμάτων 4 εκτελέσεων για την άσκηση αυτή με αριθμούς νημάτων αυτούς που φαίνονται, ποσοστό 90% αναζήτησης και 10% εισαγωγής.

